

ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA

2º Ingeniería Matemática e Inteligencia Artificial



PROYECTO FINAL

PRESENTADO POR:

Álvaro Bilbao Pardo

Asignatura: Bases de Datos

Curso 2025 – 2026

Profesores:

Israel Alonso Martínez y Pablo Sánchez Pérez

Glosario

ASIN (Amazon Standard Identification Number)

Identificador único asignado por Amazon a cada producto de su catálogo. En este proyecto se utiliza como clave primaria de la tabla ARTICULO.

Correlación de Pearson

Medida estadística que cuantifica el grado de relación lineal entre dos variables. En el proyecto se utiliza para medir la similitud de comportamiento entre dos usuarios a partir de las valoraciones que han dado a artículos en común.

Clave foránea (Foreign Key)

Campo o conjunto de campos de una tabla que hacen referencia a la clave primaria de otra tabla, garantizando la integridad referencial entre ambas.

Clave primaria compuesta

Clave primaria formada por más de un campo. En este proyecto la tabla REVIEW utiliza la combinación (reviewer_id, asin, unix_review_time) para identificar de forma única cada reseña.

JSON (JavaScript Object Notation)

Formato ligero de intercambio de datos basado en pares clave-valor. Es el formato de los ficheros de reseñas de Amazon utilizados en el proyecto.

MongoDB

Sistema de gestión de bases de datos documental NoSQL que almacena los datos como documentos JSON (BSON internamente). En este proyecto se utiliza para almacenar los campos semiestructurados de las reseñas.

MySQL

Sistema de gestión de bases de datos relacional de código abierto. En este proyecto almacena los datos estructurados del dominio (usuarios, artículos y reseñas).

Neo4J

Sistema de gestión de bases de datos de grafos que almacena los datos como nodos y relaciones. Se utiliza en la tercera parte del proyecto para representar similitudes y conexiones entre usuarios y artículos.

WordCloud (nube de palabras)

Representación visual de la frecuencia de palabras en un texto, donde el tamaño de cada palabra es proporcional a su frecuencia de aparición.

Índice

Glosario	1
1 Introducción	3
2 Primera parte: Diseño y carga de datos	4
2.1 Esquema relacional de MySQL	4
2.2 Distribución de datos entre MySQL y MongoDB	5
2.3 Implementación de la carga de datos	6
2.4 Problemas encontrados y soluciones	7
3 Segunda parte: Menú de visualización	8
3.1 Estructura general del menú	8
3.2 Consulta 1 — Evolución de reviews por año	9
3.3 Consulta 2 — Evolución de la popularidad de artículos	10
3.4 Consulta 3 — Histograma por nota	11
3.5 Consulta 4 — Evolución de reviews a lo largo del tiempo	12
3.6 Consulta 5 — Histograma de reviews por usuario	13
3.7 Consulta 6 — Nube de palabras por categoría	14
3.8 Consulta 7 — Usuarios únicos por categoría	15
4 Tercera parte: Neo4J	16
4.1 Configuración de la conexión	16
4.2 Opción 1 — Similitudes entre usuarios (correlación de Pearson)	17
4.3 Opción 2 — Enlaces entre usuarios y artículos aleatorios	19
4.4 Opción 3 — Usuarios que han puntuado más de un tipo de artículo	20
4.5 Opción 4 — Artículos populares y artículos en común	21
5 Cuarta parte: Inserción de un nuevo dataset	23
6 Quinta parte: Más visualización y relación con Machine Learning	24
6.1 Visualizaciones con Tableau	24
6.2 Descripción del modelo de recomendación	26
6.3 Implementación de la recomendación por popularidad	27
7 Partes opcionales	28
8 Conclusiones	29

Introducción

El siguiente documento consiste en el Proyecto final de la asignatura de Bases de Datos del grado en Ingeniería Matemática e Inteligencia Artificial (iMat) de la Universidad Pontificia Comillas (ICAI).

El objetivo de este proyecto es construir un sistema de gestión y análisis de datos a partir de los conjuntos de reseñas de Amazon, combinando una base de datos relacional (MySQL), una base de datos documental (MongoDB) y una base de datos de grafos (Neo4J). Se ha desarrollado, además, una aplicación en Python con menús de visualización interactivos y herramientas de análisis.

Los datasets utilizados son los siguientes:

- Toys and Games 5-core (Toys_and_Games_5.json) — 167 597 reseñas
- Video Games 5-core (Video_Games_5.json) — 231 780 reseñas
- Digital Music 5-core (Digital_Music_5.json) — 64 706 reseñas
- Musical Instruments 5-core (Musical_Instruments_5.json) — 10 261 reseñas

Cada línea de estos ficheros es un objeto JSON con la siguiente estructura:

```
{"reviewerID": "A2HD75EMZR8QLN", "asin": "0700099867",  
"reviewerName": "123", "helpful": [8, 12],  
"reviewText": "...", "overall": 5, "summary": "...",  
"unixReviewTime": 1341792000, "reviewTime": "07 9, 2012"}
```

El proyecto se ha estructurado en cinco partes obligatorias, siguiendo las instrucciones que se indican en el enunciado. A lo largo de esta memoria se detalla el razonamiento seguido en cada una de ellas, los problemas encontrados y las soluciones escogidas.

Primera parte: Diseño y carga de datos

Esquema relacional de MySQL

El punto de partida del diseño es el análisis de las relaciones que existen entre los datos del dataset. Al estudiar la estructura de cada reseña se identifican dos entidades principales y una relación entre ellas:

• Relaciones entre entidades:

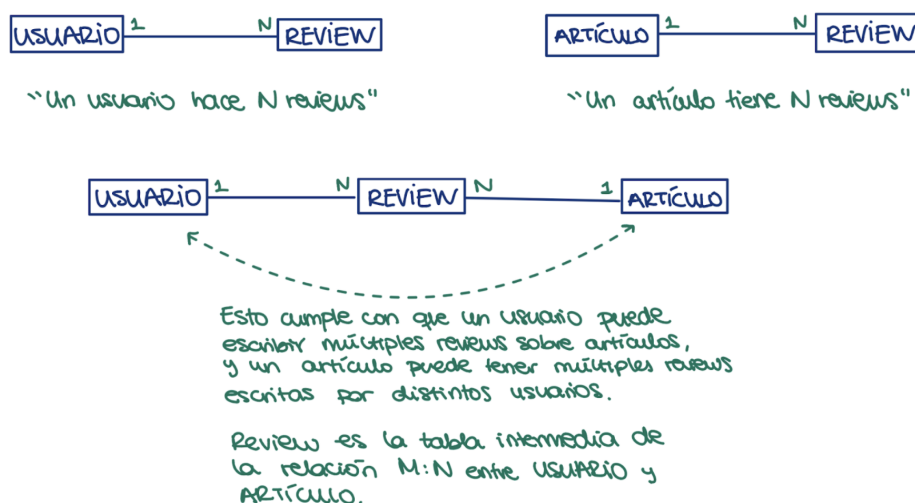


Figura 1: Representación de las relaciones entre entidades

- Un usuario puede escribir múltiples reseñas sobre distintos artículos.
- Un artículo puede recibir múltiples reseñas de distintos usuarios.
- La reseña es la relación N:M entre USUARIO y ARTÍCULO, y porta sus propios atributos.

A partir de este análisis, el esquema relacional queda definido con tres tablas:

• Esquema relacional:

USUARIO (reviewer_id, reviewer_name)
PRIMARY KEY

ARTÍCULO (asin, tipo)
PRIMARY KEY

REVIEW (reviewer_id, asin, unix_review_time, overall, review_time)
PRIMARY KEY

reviewer_id FK REFERS USUARIO (reviewer_id)

asin FK REFERS ARTÍCULO (asin)

Figura 2: Representación del esquema relacional

La clave primaria de REVIEW es compuesta por tres campos porque un mismo usuario puede realizar varias reseñas sobre un mismo artículo en momentos distintos. La combinación `reviewer_id` + `asin` no sería suficiente para identificar de forma única cada reseña, por lo que es necesario añadir `unix_review_time`, que garantiza la unicidad incluso en esos casos.

Distribución de datos entre MySQL y MongoDB

El diseño sigue un enfoque híbrido en el que todos los campos del dataset quedan almacenados entre las dos bases de datos. La distribución responde a la naturaleza de cada campo:

Datos almacenados en MySQL

Se almacenan en MySQL los campos estructurados, es decir, aquellos que tienen un tipo de dato fijo, participan en relaciones claras entre tablas y son útiles para consultas relacionales con JOIN, agrupaciones y filtros:

- `reviewerID` → tabla USUARIO
- `reviewerName` → tabla USUARIO
- `asin` → tabla ARTICULO
- `tipo` (inferido del fichero de origen) → tabla ARTICULO
- `overall` → tabla REVIEW
- `unixReviewTime` → tabla REVIEW
- `reviewTime` (parseado a formato DATE) → tabla REVIEW

El campo `reviewTime` original tiene el formato “mes día, año” (por ejemplo, “07 9, 2012”), que no es directamente compatible con el tipo DATE de MySQL. Para resolverlo se implementó la función «`pasar_fecha()`», que transforma esa cadena al formato «YYYY-MM-DD», tal y como exige MySQL.

Datos almacenados en MongoDB

Se almacenan en MongoDB los campos semiestructurados o con contenido variable, que no encajan bien en el modelo tabular:

- `reviewText` — texto libre de longitud variable
- `summary` — texto libre de longitud variable
- `helpful` — array de dos enteros [útil, total]

Cada documento de MongoDB incluye además `reviewer_id`, `asin` y `unix_review_time` como campos de enlace con MySQL, lo que permite combinar datos de ambas bases de datos en las consultas que lo requieran.

Este diseño aprovecha las fortalezas de cada sistema, ya que MySQL gestiona entidades, relaciones y atributos estructurados con toda la potencia del modelo relacional, mientras que MongoDB gestiona texto libre y arrays de forma natural y flexible, sin necesidad de descomponer su contenido en columnas adicionales.

Implementación de la carga de datos

La carga de datos se implementa en dos ficheros Python: «configuracion.py» y «load_data.py».

configuracion.py

Este fichero centraliza todos los parámetros de conexión y las rutas a los ficheros de datos, de modo que se puedan modificar credenciales y rutas sin tocar ningún otro archivo:

```
4  #MYSQL
5  host = "localhost"
6  user = "root"
7  password = "alvaro123"
8  db_sql = "ProyectoBasesSQL"
```

Figura 3: Extracto del código de «configuracion.py»

```
10 #MONGODB
11 mongo_uri = "mongodb://localhost:27017"
12 db_mongo = "ProyectoBasesMongo"
13 coleccion = "Reviews"
```

Figura 4: Extracto 2 del código de «configuración.py»

```
15 #RUTA FICHEROS
16 ruta_toys_and_games = "Toys_and_Games_5.json"
17 ruta_video_games = "Video_Games_5.json"
18 ruta_digital_music = "Digital_Music_5.json"
19 ruta_musical_instruments = "Musical_Instruments_5.json"
```

Figura 5: Extracto 3 del código de «configuración.py»

load_data.py

Este fichero contiene toda la lógica de creación e inicialización de la infraestructura de datos. Al ejecutarlo se realizan las siguientes operaciones en orden:

- `crear_base_datos_mysql()`: crea la base de datos en MySQL si no existe, utilizando los parámetros de «configuracion.py».
- `crear_tablas_mysql()`: elimina las tablas existentes (para evitar conflictos con ejecuciones anteriores) y las recrea con sus claves primarias y foráneas.
- `preparar_mongodb()`: elimina la colección Reviews de MongoDB para comenzar la carga desde cero y evitar duplicados.

- `cargar_fichero(ruta, tipo)`: procesa cada fichero de datos línea a línea, evitando cargar el fichero completo en memoria. Por cada línea se extraen los campos necesarios, se insertan los datos estructurados en MySQL y los datos textuales en MongoDB.

La función «`cargar_fichero`» es el núcleo de la carga. Recibe la ruta al fichero JSON y el tipo de producto correspondiente, y por cada reseña realiza tres inserciones en MySQL (USUARIO, ARTICULO y REVIEW) más una inserción en MongoDB con el texto, resumen y valoraciones de utilidad. El parámetro `tipo` permite etiquetar cada artículo con su categoría para facilitar las consultas posteriores.

El bloque principal del programa llama a estas funciones en el orden correcto, de forma que ejecutar «`load_data.py`» reconstruye la base de datos completa desde cero:

```
126 if __name__ == "__main__":
127     crear_base_datos_mysql()
128     crear_tablas_mysql()
129     preparar_mongodb()
130
131     cargar_fichero(ruta_toys_and_games, "Toys and Games")
132     cargar_fichero(ruta_video_games, "Video Games")
133     cargar_fichero(ruta_digital_music, "Digital Music")
134     cargar_fichero(ruta_musical_instruments, "Musical Instruments")
```

Figura 6: Extracto del main de «`load_data.py`»

Problemas encontrados y soluciones

Clave primaria duplicada en USUARIO y ARTICULO

Al procesar los ficheros, un mismo usuario o un mismo artículo pueden aparecer en múltiples reseñas. Al intentar insertar un registro con una clave primaria que ya existe, MySQL lanza un error. La solución adoptada fue utilizar la cláusula «ON DUPLICATE KEY UPDATE» en las sentencias «INSERT», de modo que si ya existe un registro con esa clave primaria, en lugar de generar un error, MySQL actualiza los campos no clave. Esto garantiza que cada usuario y cada artículo se almacenen una única vez y que la integridad referencial con la tabla REVIEW se mantenga en todo momento:

```
76 sql_usuario = """INSERT INTO USUARIO (reviewer_id, reviewer_name)
77 VALUES (%s, %s)
78 ON DUPLICATE KEY UPDATE reviewer_name = VALUES(reviewer_name)"""
```

Figura 7: Extracto código «`load_data.py`»

Campo reviewerName ausente en algunos documentos

No todos los objetos JSON del dataset incluyen el campo `reviewerName`. Para evitar que el script falle al intentar acceder a un campo inexistente, se añadió una comprobación explícita: si el campo está presente se usa su valor, y si no, se asigna una cadena vacía. Esto se gestiona de la siguiente forma:


```

94         reviewer_id = doc["reviewerID"]
95         if "reviewerName" in doc:
96             reviewer_name = doc["reviewerName"]
97         else:
98             reviewer_name = ""

```

Figura 8: Extracto 2 código «load_data.py»

Segunda parte: Menú de visualización

Estructura general del menú

La aplicación de visualización se implementa en el fichero «menu_visualizacion.py». Al arrancar el programa se muestra un cartel decorativo, seguido de un menú de opciones que se repite en bucle hasta que el usuario selecciona la opción de salir.

Para facilitar la selección de categoría de producto, que se repite en varias consultas, se implementó una función reutilizable llamada «pedir_tipo()». Esta función recibe un parámetro booleano («permitir_todo») que controla si se ofrece la opción de mostrar datos para todas las categorías a la vez. Las categorías disponibles se gestionan mediante un diccionario clave-valor, de modo que el usuario solo tiene que introducir un número en lugar del nombre completo de la categoría, evitando errores tipográficos:

```

41 #Creamos un diccionario con los tipos de productos para poder elegir cuál queremos estudiar en las consultas
42 tipos = {"1": "Video Games", "2": "Toys and Games", "3": "Digital Music", "4": "Musical Instruments"}

```

Figura 9: Extracto código «menu_visualizacion.py»

La función «conectar_mysql()» y «conectar_mongodb()» centralizan la lógica de conexión a cada base de datos, siguiendo el mismo patrón de «configuracion.py» para los parámetros.



Figura 10: Imagen del cartel + Menú de Visualización al ejecutar «menu_visualizacion.py»

Consulta 1 — Evolución de reviews por año

Esta consulta muestra un histograma con el número de reseñas agrupadas por año. Tras seleccionar una categoría (o la opción «todo» para ver todos los productos en conjunto), se lanza una consulta SQL que extrae el año a partir del campo «review_time» usando la función «YEAR()», agrupa los resultados por año y los ordena cronológicamente:

```
83 sql = """
84     SELECT YEAR(r.review_time) AS ano, COUNT(*)
85     FROM REVIEW r
86     JOIN ARTICULO a ON r.asin = a.asin
87     WHERE a.tipo = %s
88     GROUP BY ano
89     ORDER BY ano
90 """
```

Figura 11: Primera consulta de MySQL

Los resultados se almacenan en dos listas (años y número de reseñas) y se representan con un gráfico de barras de matplotlib, con las etiquetas del eje X rotadas 45 grados para mejorar la legibilidad.

```
Elige una opción: 1
La evolución de reviews por años
0 - Todo
1 - Video Games
2 - Toys and Games
3 - Digital Music
4 - Musical Instruments
s - salir
Selecciona una categoría:0
```

Figura 12: Ejemplo consulta 1 a través de la interfaz de «menu_visualizacion.py»

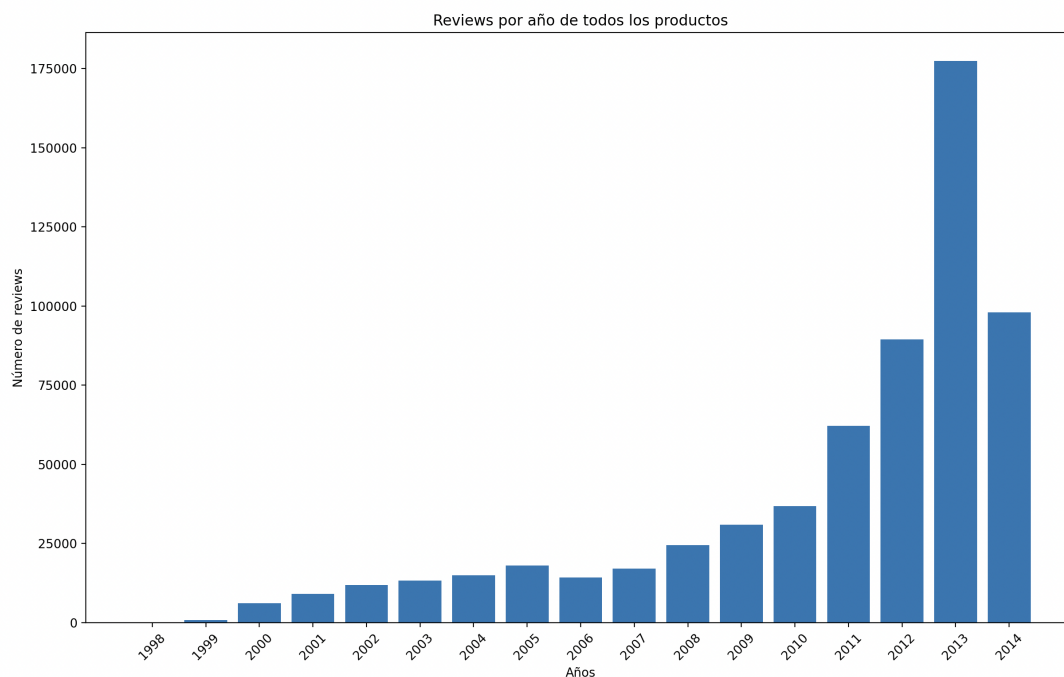


Figura 13: Gráfico resultante tras la primera consulta en «menu_visualizacion.py»

Consulta 2 — Evolución de la popularidad de artículos

Esta consulta genera una curva que muestra los artículos ordenados de mayor a menor popularidad, siendo la popularidad el número de reseñas que tiene cada artículo. La consulta SQL agrupa las reseñas por «asin» y las ordena de forma descendente por conteo:

```
133 sql = ""
134     SELECT r.asin, COUNT(*)
135     FROM REVIEW r
136     JOIN ARTICULO a ON r.asin = a.asin
137     WHERE a.tipo = %s
138     GROUP BY r.asin
139     ORDER BY COUNT(*) DESC
140 ""
```

Figura 14: Segunda consulta MySQL

El resultado se representa como una línea donde el eje X es la posición relativa de cada artículo (de 0 al total) y el eje Y es su número de reseñas. La curva resultante tiene una forma característica de cola larga, mostrando que unos pocos artículos concentran la mayor parte de las reseñas mientras que la gran mayoría tiene muy pocas.

```
Elige una opción: 2
Evolución de la popularidad de los artículos
0 - Todo
1 - Video Games
2 - Toys and Games
3 - Digital Music
4 - Musical Instruments
s - salir
Selecciona una categoría:0
```

Figura 15: Ejemplo consulta 2 a través de la interfaz

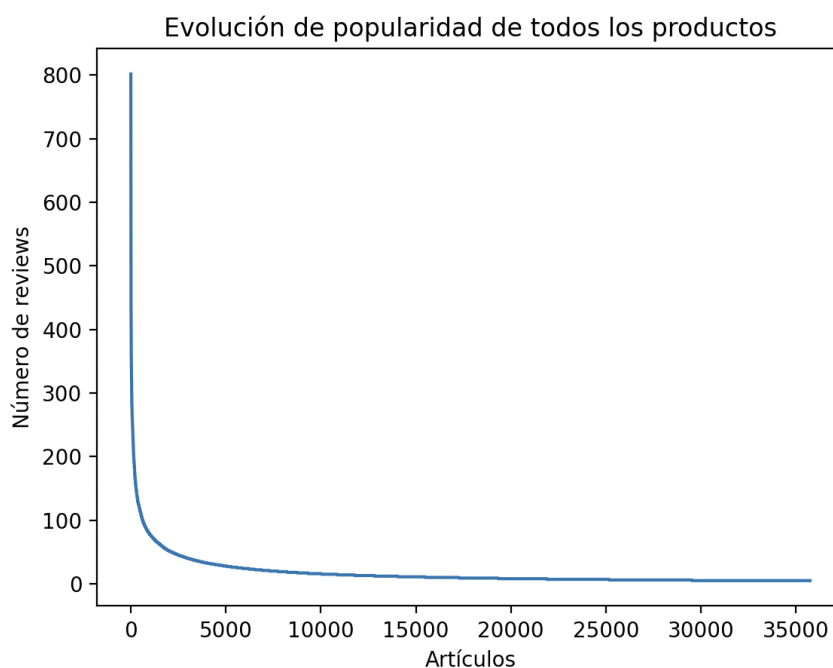


Figura 16: Gráfico resultante tras la segunda consulta

Consulta 3 — Histograma por nota

Esta consulta ofrece al usuario dos modos de análisis. El primero agrupa las reseñas por nota global (valores 1 a 5) para un tipo de producto o para todos. El segundo permite buscar las notas de un artículo concreto introduciendo su ASIN.

Para el modo de artículo individual se comprueba primero que el ASIN introducido existe en la base de datos antes de ejecutar la consulta, mostrando un mensaje de error claro si no se encuentra:

```
204         asin = input("Introduce el asin del artículo:").strip()
205
206         cursor.execute("SELECT asin FROM ARTICULO WHERE asin = %s", (asin,))
207
208         if cursor.fetchone() is None: #Comprobamos que exista el artículo
209             print(f"El artículo «{asin}» no existe en la base de datos.")
```

Figura 17: Tercera consulta MySQL

El histograma resultante muestra en el eje X los valores de nota (1, 2, 3, 4 y 5) y en el eje Y el número de reseñas con cada puntuación, lo que permite visualizar rápidamente la distribución de valoraciones.

```
Elige una opción: 3
Histograma por nota
1 - Por tipo de producto (o todos)
2 - Por artículo individual (asin)
c - Cancelar
Selecciona:1
0 - Todo
1 - Video Games
2 - Toys and Games
3 - Digital Music
4 - Musical Instruments
s - salir
Selecciona una categoría:0
```

Figura 18: Ejemplo consulta 3 a través de la interfaz



Figura 19: Gráfico resultante tras la tercera consulta

Consulta 4 — Evolución de reviews a lo largo del tiempo

Esta consulta muestra la evolución acumulada del número de reseñas a lo largo del tiempo para una categoría concreta. En esta opción no se ofrece la posibilidad de ver «todo» porque el enunciado pide analizar cada tipo por separado, por lo que «pedir_tipo()» se llama con «permitir_todo=False».

La consulta SQL recupera los `unix_review_time` de todas las reseñas de la categoría seleccionada ordenados cronológicamente.

```
269      sql = ""
270      SELECT r.unix_review_time
271      FROM REVIEW r
272      JOIN ARTICULO a ON r.asin = a.asin
273      WHERE a.tipo = %s
274      ORDER BY r.unix_review_time
275      ""
```

Figura 20: Cuarta consulta MySQL

A partir de esa lista se construye un vector acumulado donde cada posición «i» contiene el valor «i+1», representando el número total de reseñas registradas hasta ese momento. El eje X es el timestamp Unix y el eje Y el número acumulado de reseñas.

```
Elige una opción: 4
Evolución de reviews a lo largo del tiempo
0 - Todo
1 - Video Games
2 - Toys and Games
3 - Digital Music
4 - Musical Instruments
s - salir
Selecciona una categoría:0
```

Figura 21: Ejemplo consulta 4 a través de la interfaz

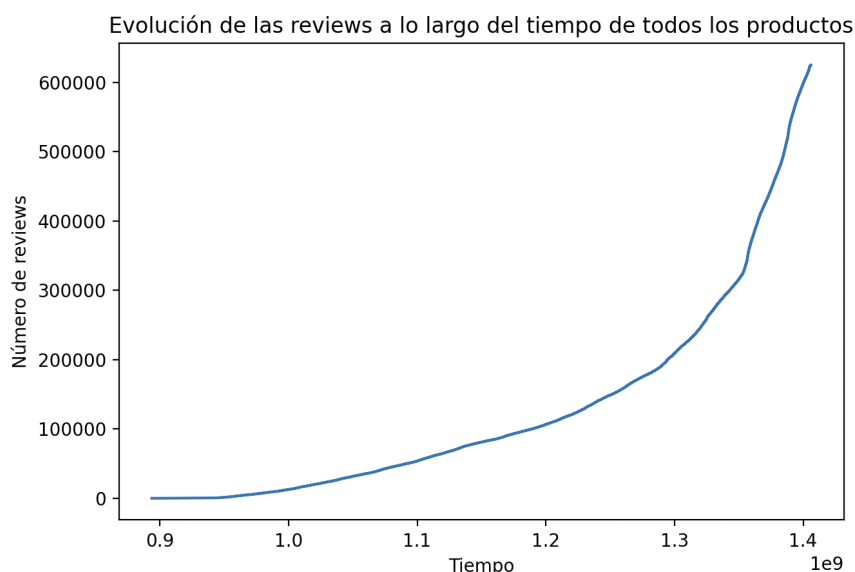


Figura 22: Gráfico resultante tras la cuarta consulta

Consulta 5 — Histograma de reviews por usuario

Esta consulta no distingue por tipo de producto. Agrupa todas las reseñas por «reviewer_id» y cuenta cuántas ha hecho cada usuario.

```
306      sql = ""
307      SELECT reviewer_id, COUNT(*)
308      FROM REVIEW
309      GROUP BY reviewer_id
310      ""
```

Figura 23: Quinta consulta MySQL

El resultado se representa como un histograma donde el eje X es el número de reseñas y el eje Y el número de usuarios que han hecho exactamente esa cantidad.

```
Elige una opción: 5
Histograma de reviews por usuario
```

Figura 24: Ejemplo consulta 5 a través de la interfaz

Al generar el histograma con el número de bins por defecto, la forma resultante difería ligeramente de la figura de referencia proporcionada en el enunciado. Tras varias pruebas se encontró que el valor de 800 bins produce el gráfico más parecido al ejemplo, así que se fijó ese valor.

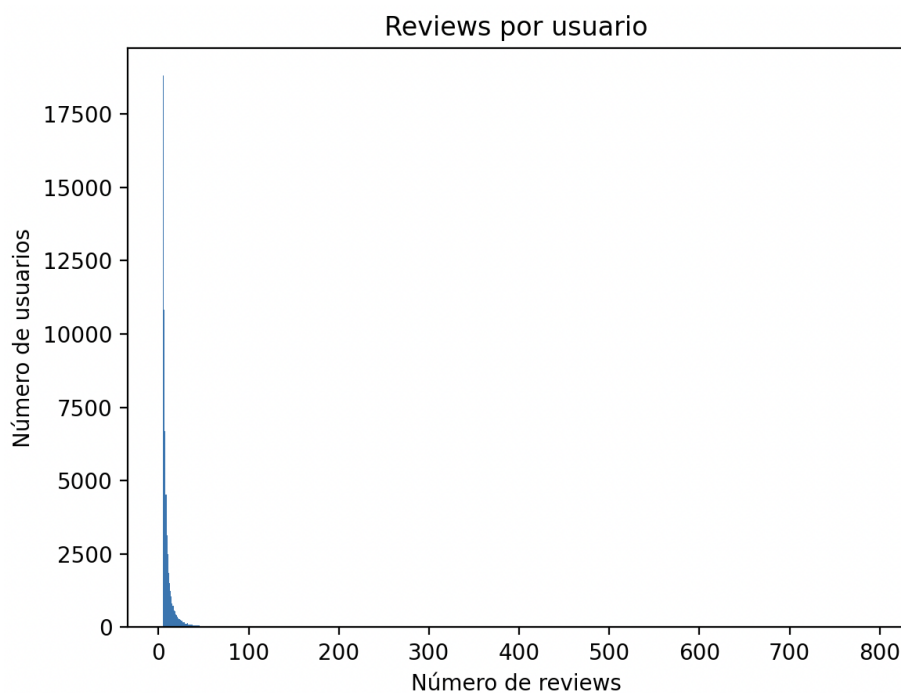


Figura 25: Gráfico resultante tras la quinta consulta

Consulta 6 — Nube de palabras por categoría

Esta consulta genera una nube de palabras a partir de los resúmenes (campo «summary») de las reseñas de una categoría. Al no almacenarse el texto en MySQL, la consulta combina ambas bases de datos: primero se obtienen los ASINs de la categoría seleccionada desde MySQL, y luego se recuperan los documentos correspondientes de MongoDB filtrando por esos ASINs:

```

352 documentos = coleccion.find({"asin": {"$in": asins}}, {"summary": 1, "_id": 0})
353
354 texto = ""
355
356 for doc in documentos:
357     if "summary" in doc:
358         resumen = doc["summary"]
359
360         palabras = resumen.split(" ")
361         for palabra in palabras:
362             if len(palabra) > 3:
363                 texto += palabra + " "
364
365 wc = WordCloud().generate(texto)

```

Figura 26: Sexta consulta MySQL

Para filtrar conectores y palabras poco significativas, solo se tienen en cuenta las palabras con más de tres caracteres. El texto resultante se pasa a la librería «wordcloud» para generar la imagen. Para poder utilizar esta librería ha sido necesario descargarsela utilizando el comando «pip install wordcloud».

```

Elige una opción: 6
Nube de palabras por categoría
1 - Video Games
2 - Toys and Games
3 - Digital Music
4 - Musical Instruments
s - salir
Selecciona una categoría:4

```

Figura 27: Ejemplo consulta 6 a través de la interfaz

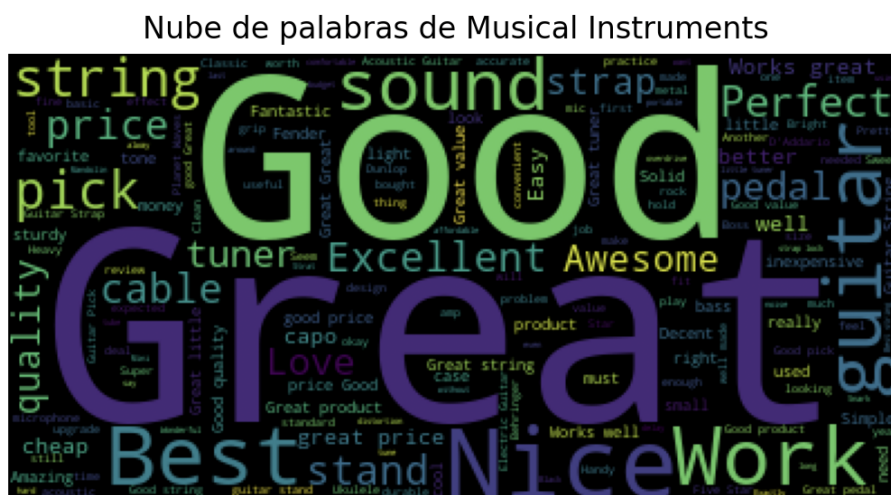


Figura 28: Gráfico resultante tras la sexta consulta

Consulta 7 — Usuarios únicos por categoría

Como gráfica libre se optó por representar el número de usuarios únicos por categoría de producto. A diferencia de las consultas anteriores, que se centran en el volumen de reseñas, este gráfico mide el alcance de cada categoría en términos de participación real de usuarios distintos, aportando una perspectiva complementaria sobre el comportamiento de los consumidores.

La consulta SQL utiliza «COUNT(DISTINCT reviewer_id)» agrupando así por tipo de artículo:

```
380      sql = ""
381      SELECT a.tipo, COUNT(DISTINCT r.reviewer_id)
382      FROM REVIEW r
383      JOIN ARTICULO a ON r.asin = a.asin
384      GROUP BY a.tipo
385      ORDER BY COUNT(DISTINCT r.reviewer_id) DESC
386      ""
```

Figura 29: Séptima consulta MySQL

```
Elige una opción: 7
Usuarios únicos por categoría
```

Figura 30: Ejemplo consulta 7 a través de la interfaz

El resultado se representa como un gráfico de barras con las categorías en el eje X y el número de usuarios únicos en el eje Y.

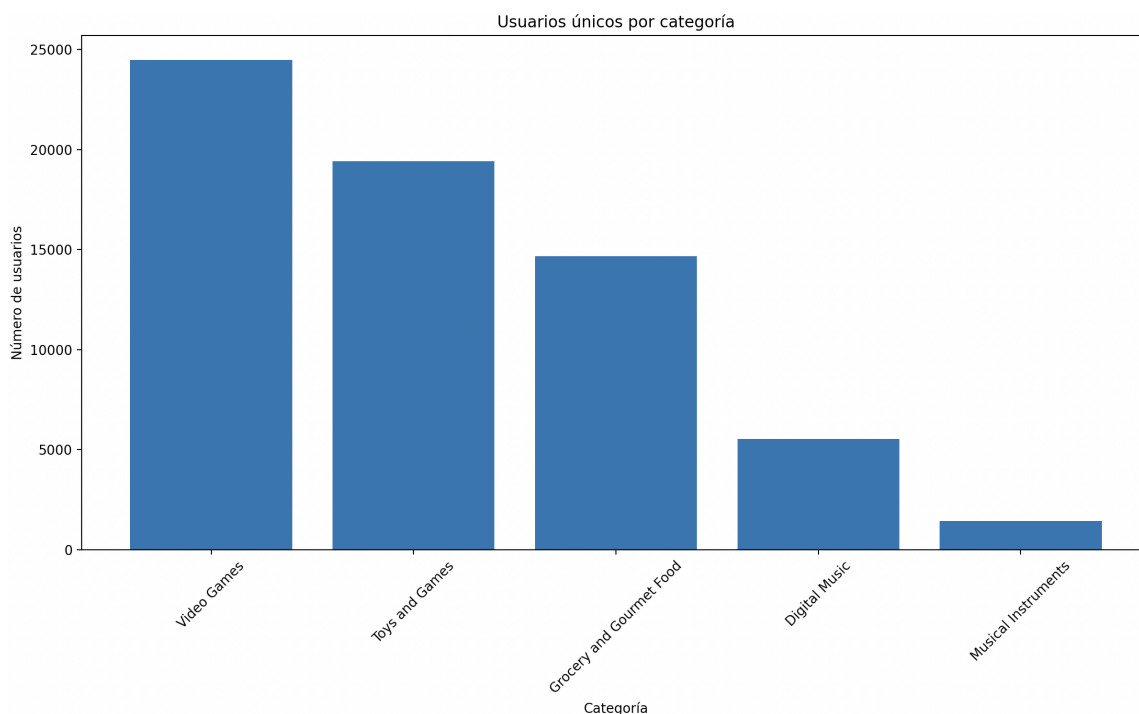


Figura 31: Gráfico resultante tras la séptima consulta

Tercera parte: Neo4J

Configuración de la conexión

Para esta parte se creó el fichero «neo4JProyecto.py», que contiene un menú con cuatro opciones de análisis sobre grafos. La conexión a Neo4J se configura en «configuracion.py», añadiendo los siguientes parámetros:

```
21  #NEO4J
22  neo4j_uri = "neo4j://localhost:7687"
23  neo4j_user = "neo4j"
24  neo4j_password = "contrasena"
```

Figura 32: Extracto código «configuracion.py»

La función «conectar_neo4j()» crea un driver de Neo4J usando esas credenciales. La función «limpiar_neo4j(driver)» borra todos los nodos y relaciones existentes antes de cada operación, garantizando que los datos de apartados anteriores no interfieran con los nuevos:

```
43  #Borramos todo lo que haya en Neo4j para empezar de cero
44  def limpiar_neo4j(driver):
45      with driver.session() as session:
46          session.run("MATCH (n) DETACH DELETE n")
```

Figura 33: Extracto código «neo4JProyecto.py»

El fichero incluye también el cartel decorativo del proyecto y el bucle de menú con gestión de excepciones, de modo que cualquier error durante la ejecución de una opción se informa por pantalla sin interrumpir el programa.



Figura 34: Cartel decorativo + Menu Neo4J en «neo4JProyecto.py»

Opción 1 — Similitudes entre usuarios (correlación de Pearson)

Esta opción calcula la similitud entre los n usuarios con más reseñas utilizando la correlación de Pearson. El valor de n se pide al usuario por pantalla (el enunciado recomienda 30 como referencia) y puede cambiarse libremente en cualquier ejecución.

El proceso es el siguiente:

- «obtener_top_usuarios(n)»: consulta MySQL para obtener los n usuarios con más reseñas, ordenados de forma descendente por conteo.
- «obtener_reviews_usuario($reviewer_id$)»: recupera todas las reseñas de un usuario como un diccionario {asin: overall}.
- «calcular_pearson($reviews_u$, $reviews_v$)»: calcula la correlación de Pearson entre los dos usuarios. Solo se tienen en cuenta los artículos puntuados en común. Si no hay artículos en común, la función devuelve «None» y no se crea enlace en Neo4J.
- «calcular_similitudes($lista$)»: itera sobre todos los pares posibles de usuarios y calcula la similitud para cada par.
- «cargar_similitudes_neo4j($usuarios$, $similitudes$)»: crea nodos USUARIO en Neo4J y relaciones SIMILITUD bidireccionales con el valor de la correlación como propiedad.

La fórmula de Pearson implementada es:

$$PC(u, v) = \frac{\sum(r_{ui} - r_u)(r_{vi} - r_v)}{\sqrt{\sum(r_{ui} - r_u)^2 \cdot \sum(r_{vi} - r_v)^2}}$$

Donde r_u y r_v son las medias de valoraciones de cada usuario sobre el total de sus artículos, y la suma se realiza únicamente sobre los artículos puntuados en común.

Al finalizar, se realiza una consulta Cypher sobre Neo4J para mostrar el usuario con más vecinos (mayor número de relaciones SIMILITUD):

```
202 with driver.session() as session:
203     resultado = session.run("""
204         MATCH (u:USUARIO)-[:SIMILITUD]->(v:USUARIO)
205         WITH u, COUNT(v) AS vecinos
206         ORDER BY vecinos DESC
207         LIMIT 1
208         RETURN u.reviewer_id AS usuario, vecinos
209     """)
```

Figura 35: Extracto código «neo4JProyecto.py»

```
Elige una opción:1
Similitudes entre usuarios con correlación de Pearson
¿Cuántos usuarios con más reviews quieres analizar?:30
Usuarios obtenidos:
30
Similitudes calculadas:
282

Usuario con más vecinos:
A2582KMXLK2P06
Número de vecinos:
29

Datos cargados en Neo4j, están en http://localhost:7474
```

Figura 36: Ejemplo opción 1 a través de la interfaz «neo4JProyecto.py»

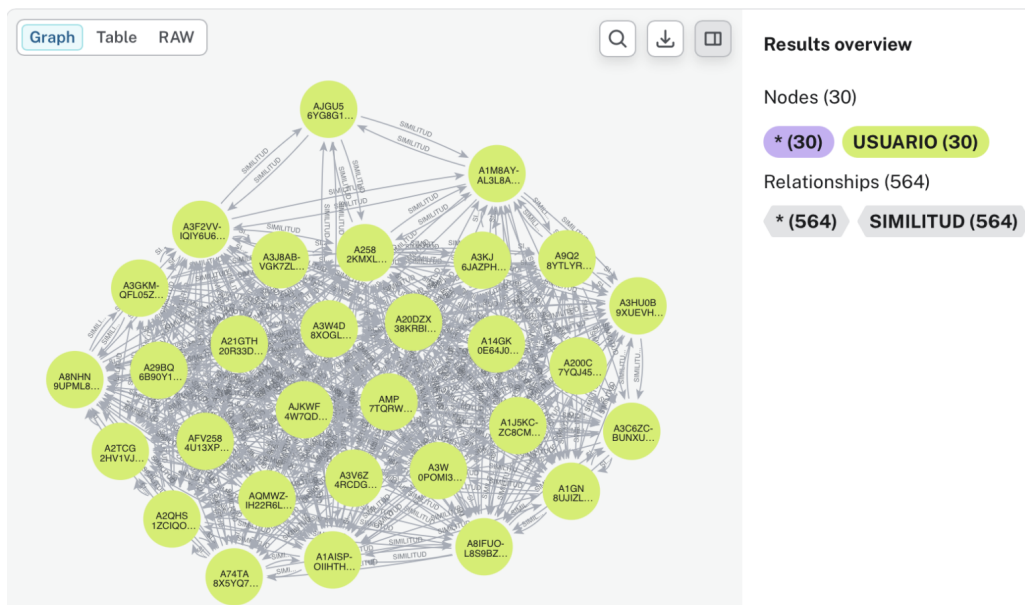


Figura 37: Gráfico resultante tras la opción 1 «neo4JProyecto.py»

Opción 2 — Enlaces entre usuarios y artículos aleatorios

Esta opción selecciona de forma aleatoria un número de artículos de una categoría determinada (ambos parámetros se solicitan al usuario por pantalla) y crea en Neo4J nodos ARTICULO y nodos USUARIO para todos los usuarios que hayan puntuado esos artículos. La relación PUNTUA incluye como propiedades la nota (overall) y el momento de la reseña (unix_review_time).

Antes de cargar los datos se llama a «limpiar_neo4j()» para partir de una base de datos vacía, evitando conflictos con los datos del apartado anterior. Cuando la carga termina, se imprime un mensaje indicando la URL de Neo4J Browser para que el usuario pueda consultar el grafo.

```
Elige una opción:2
Enlaces entre usuarios y artículos aleatorios
1 - Video Games
2 - Toys and Games
3 - Digital Music
4 - Musical Instruments
s - salir
Selecciona una categoría: 1
¿Cuántos artículos aleatorios quieres coger?:5
Artículos cargados en Neo4j correctamente
Puedes verlo en http://localhost:7474
```

Figura 38: Ejemplo opción 2 a través de la interfaz

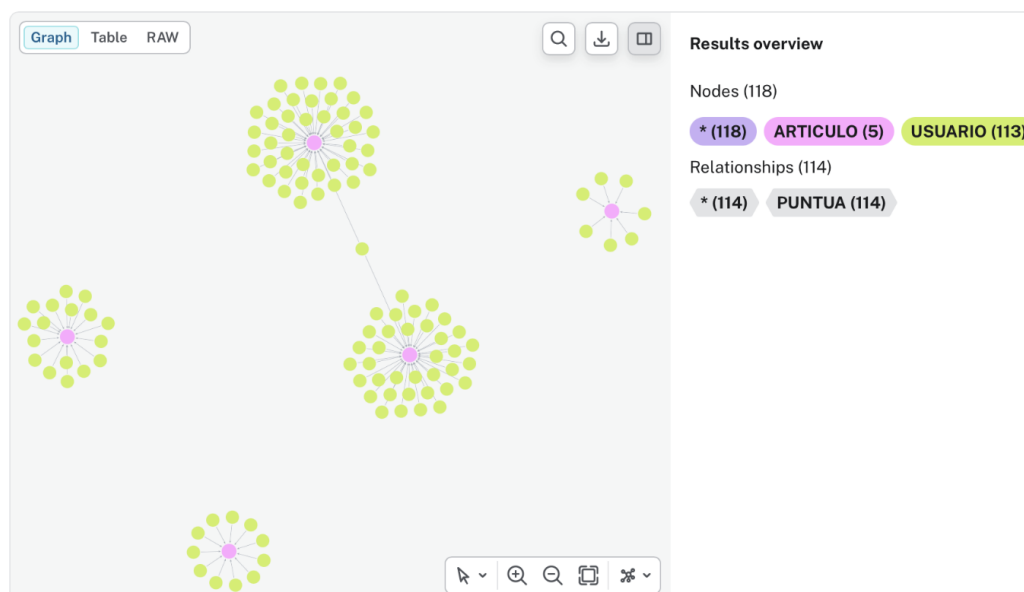


Figura 39: Gráfico resultante tras la opción 2

Opción 3 — Usuarios que han puntuado más de un tipo de artículo

Esta opción recupera los primeros 400 usuarios ordenados alfabéticamente por nombre de MySQL y filtra aquellos que han puntuado artículos de al menos dos tipos distintos (por ejemplo, un usuario que haya valorado tanto videojuegos como instrumentos musicales).

Para cada usuario que cumple el criterio, se crean en Neo4J nodos USUARIO y nodos TIPO (uno por cada categoría distinta consumida), conectados mediante relaciones CONSUME cuya propiedad cantidad indica el número de artículos de ese tipo que ha puntuado el usuario.

La consulta SQL que determina los tipos consumidos por cada usuario es:

```
391      sql = ""
392      SELECT a.tipo, COUNT(*)
393      FROM REVIEW r
394      JOIN ARTICULO a ON r.asin = a.asin
395      WHERE r.reviewer_id = %s
396      GROUP BY a.tipo
397      ""
```

Figura 40: Consulta MySQL, tipos consumidos por cada usuario

```
Elige una opción:3
Usuarios que han puntuado más de un tipo de artículo
Usuarios multitypo cargados en Neo4j correctamente
Total de usuarios cargados:
22
Puedes verlo en http://localhost:7474
```

Figura 41: Ejemplo opción 3 a través de la interfaz

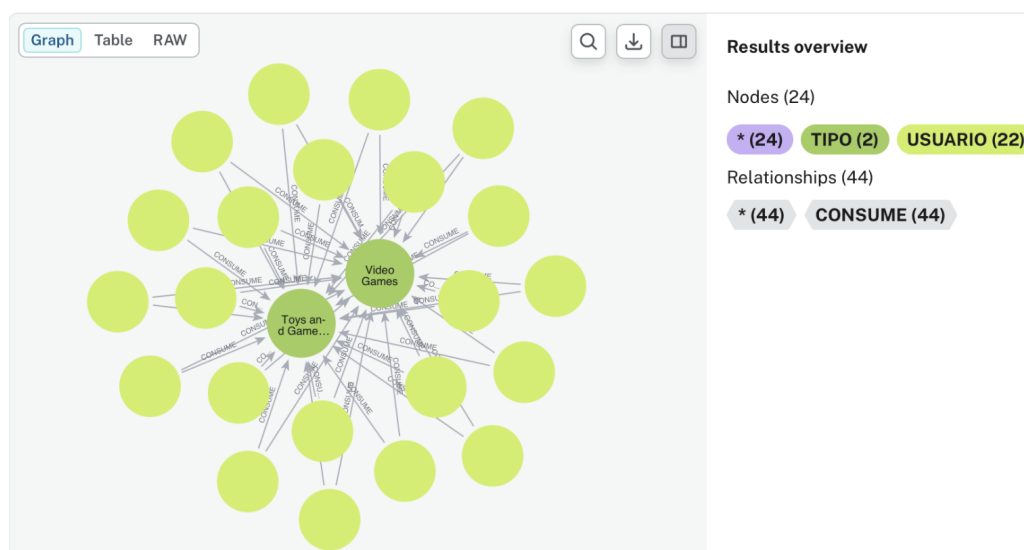


Figura 42: Gráfico resultante tras la opción 3

Opción 4 — Artículos populares y artículos en común

Esta opción selecciona los 5 artículos más populares que tienen menos de 40 reseñas (un umbral que garantiza un grafo manejable sin que los artículos sean demasiado desconocidos). La consulta SQL correspondiente es:

```
483  sql = ""
484      SELECT asin, COUNT(*)
485      FROM REVIEW
486      GROUP BY asin
487      HAVING COUNT(*) < 40
488      ORDER BY COUNT(*) DESC
489      LIMIT 5
490  ""
```

Figura 43: Consulta MySQL

Para cada uno de esos artículos se obtienen todos los usuarios que los han puntuado y se crean en Neo4J nodos ARTICULO, nodos USUARIO y relaciones PUNTUA. Adicionalmente, se calculan relaciones ARTICULOS_COMUNES entre pares de usuarios, con la propiedad cantidad indicando cuántos artículos han valorado ambos en común. Este cálculo se realiza mediante una consulta SQL de autounión sobre la tabla REVIEW:

```
528  sql = ""
529      SELECT COUNT(*)
530      FROM REVIEW r1
531      JOIN REVIEW r2 ON r1.asin = r2.asin
532      WHERE r1.reviewer_id = %s AND r2.reviewer_id = %s
533  ""
```

Figura 44: Consulta MySQL

Esta opción tarda algo más en ejecutarse que las anteriores debido al número de pares de usuarios que hay que comparar.

```
-----
Elige una opción:4

Artículos populares con menos de 40 reviews y artículos en común

Artículos encontrados:
B0085UA6YM - 39 reviews
B008A27UMG - 39 reviews
B008ACRPR6 - 39 reviews
B008CP6Q6M - 39 reviews
B008DS1M44 - 39 reviews

Datos cargados en Neo4j, están en http://localhost:7474
```

Figura 45: Ejemplo opción 4 a través de la interfaz

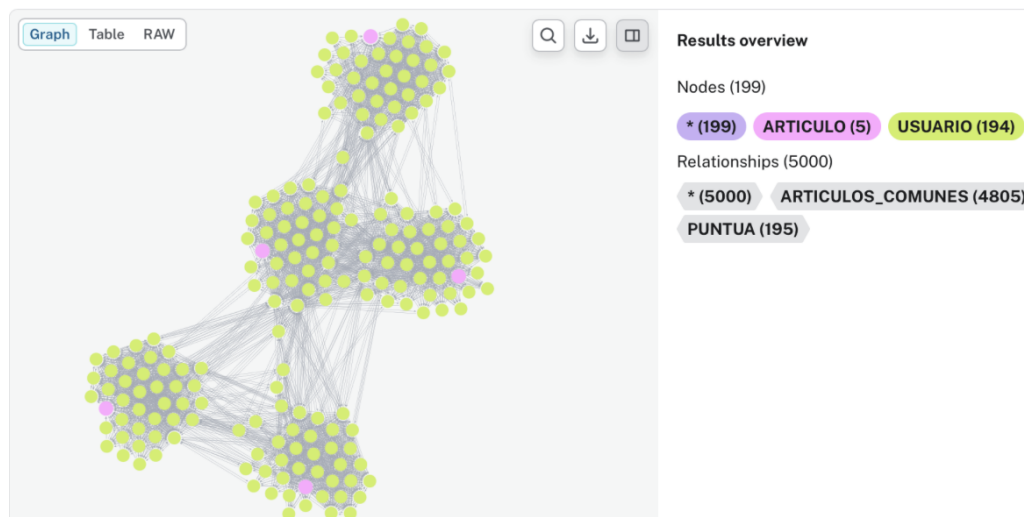


Figura 46: Gráfico resultante tras la opción 4

Cuarta parte: Inserción de un nuevo dataset

Para esta parte se seleccionó el dataset «Grocery_and_Gourmet_Food_5.json», disponible en el mismo repositorio que los cuatro originales. Este conjunto de datos sigue exactamente el mismo formato JSON que los ficheros anteriores, por lo que no fue necesario modificar el diseño de la base de datos.

Se implementó el fichero «inserta_dataset.py», que reutiliza la misma lógica de inserción de «load_data.py». La principal diferencia es que este script no recrea las tablas ni limpia MongoDB, simplemente inserta los nuevos datos sobre la infraestructura ya existente, respetando la unicidad de usuarios y artículos gracias a «ON DUPLICATE KEY UPDATE».

La ruta al nuevo fichero se añadió en «configuracion.py»:

```
26 #PARTE 4 - Ruta nuevo fichero:
27 ruta_grocery_and_gourmet_food = "Grocery_and_Gourmet_Food_5.json"
```

Figura 47: Modificación en «configuracion.py»

El bloque principal de «inserta_dataset.py» llama directamente a la función de inserción con el tipo correspondiente:

```
81 if __name__ == "__main__":
82     insertar_nuevo_dataset(ruta_grocery_and_gourmet_food, "Grocery and Gourmet Food")
```

Figura 48: Extracto del main de «inserta_dataset.py»

Este diseño garantiza que un usuario que ya existía en la base de datos por haber realizado reseñas en otras categorías se reutiliza correctamente, sin duplicados, y que sus nuevas reseñas en la categoría Grocery and Gourmet Food quedan vinculadas a ese mismo registro en la tabla USUARIO. Esto es especialmente importante dado que el enunciado indica explícitamente que un mismo usuario puede haber hecho reseñas en distintos tipos de artículos.

```
● (base) MacBook-Pro-de-ALVARO-3:Proyecto alvaro$ /opt/anaconda3/bin/python "/Users/alvaro/Desktop/ICAI/2025-2026/segundo_cuatri/bases de datos/Proyecto/inserta_dataset.py"
```

Figura 49: Captura de la carga correcta en «inserta_dataset.py»

Quinta parte: Más visualización y relación con Machine Learning

Visualizaciones con Tableau

Para esta parte se reprodujeron tres de las visualizaciones del menú Python utilizando Tableau Desktop (macOS). El procedimiento seguido fue el siguiente: se ejecutaron las consultas SQL correspondientes directamente en MySQL, se exportaron los resultados a ficheros CSV y se abrieron en Tableau como archivos de texto.

Histograma por nota

La consulta utilizada fue:

```
SELECT overall, COUNT(*) AS total_reviews  
FROM REVIEW GROUP BY overall ORDER BY overall;
```

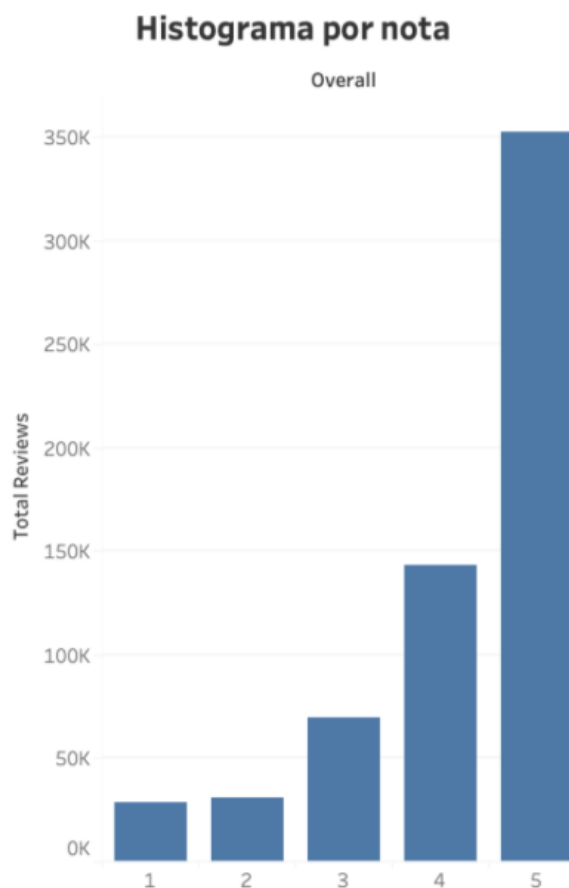


Figura 50: Gráfico Tableau - Histograma por nota

Evolución de reviews por año

La consulta utilizada fue:

```
SELECT YEAR(review_time) AS ano, COUNT(*) AS total_reviews  
FROM REVIEW GROUP BY YEAR(review_time) ORDER BY ano;
```



Figura 51: Gráfico Tableau - Evolución de reviews por año

Evolución de la popularidad de los productos

La consulta utilizada fue:

```
SELECT asin, COUNT(*) AS total_reviews  
FROM REVIEW GROUP BY asin ORDER BY total_reviews DESC;
```

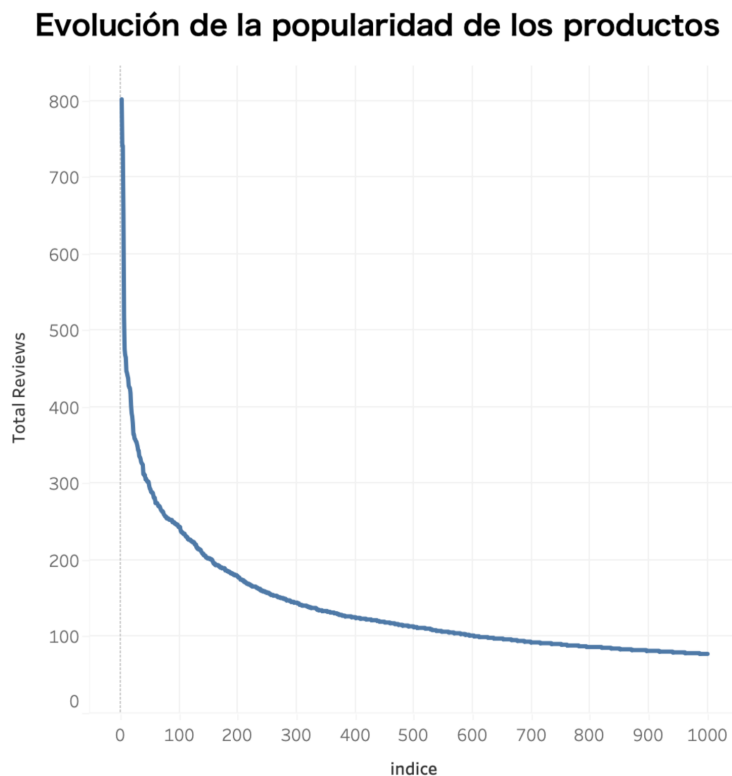


Figura 52: Gráfico Tableau - Evolución de la popularidad de los productos

Descripción del modelo de recomendación

Para diseñar un sistema de recomendación de artículos que el usuario no haya consumido previamente, se propone una fórmula de puntuación que combina tres factores complementarios:

$$\text{Puntuación}(i, u) = \text{Media}(i) + \log(1 + \text{NumReviews}(i)) + \text{BonusCategoría}(u, i)$$

Donde:

- **Media(i)** es la nota media del artículo, obtenida como `AVG(overall)` desde MySQL.
- **$\log(1 + \text{NumReviews}(i))$** es la popularidad del artículo, suavizada con logaritmo para evitar que artículos con un volumen de reseñas muy alto dominen completamente el ranking.
- **BonusCategoría(u, i)** es un incremento que refleja la afinidad del usuario con la categoría del artículo: 1 si es su categoría favorita (la que más ha consumido), 0,5 si es una categoría que también consume, y 0 si apenas ha puntuado artículos de ese tipo.

Se considera que este enfoque es sencillo de implementar con los datos disponibles y produce recomendaciones razonables sin necesidad de modelos complejos. Como extensión, podría implementarse filtrado colaborativo (utilizando las similitudes de Pearson ya calculadas en la parte de Neo4J) para personalizar aún más las recomendaciones.

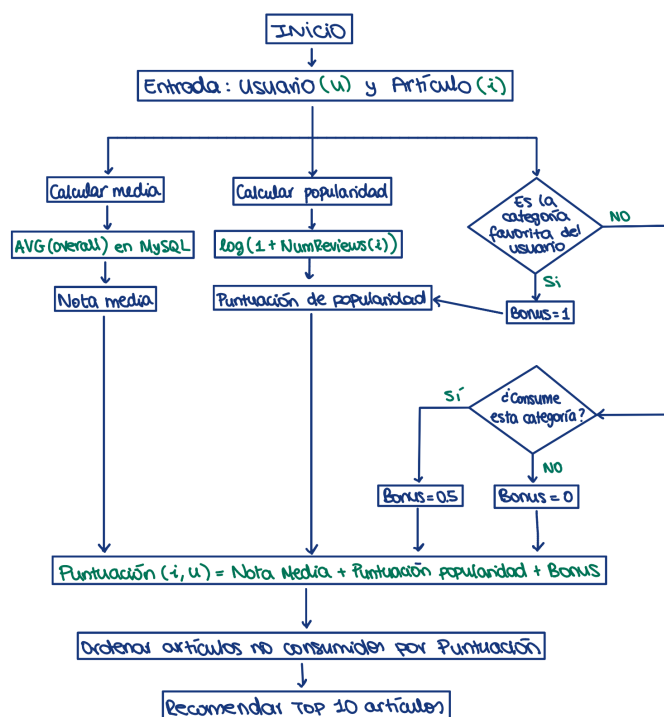


Figura 53: Diagrama del flujo del modelo de recomendación

Implementación de la recomendación por popularidad

Como implementación concreta del mecanismo de recomendación, se añadió la función «recomendar_articulos_no_consumidos()» en «menu_visualizacion.py». Esta función recibe el identificador de un usuario («reviewer_id») y un tipo de artículo, y devuelve los 10 artículos más populares de esa categoría que el usuario aún no ha valorado.

La consulta SQL que implementa esta lógica es:

```
421 sql = """
422     SELECT r.asin, COUNT(*)
423     FROM REVIEW r
424     JOIN ARTICULO a ON r.asin = a.asin
425     WHERE a.tipo = %s
426     AND r.asin NOT IN (SELECT asin FROM REVIEW WHERE reviewer_id = %s)
427     GROUP BY r.asin
428     ORDER BY COUNT(*) DESC
429     LIMIT 10
430 """
```

Figura 54: Consulta MySQL

La subconsulta «NOT IN» filtra los artículos que el usuario ya ha valorado, y el «ORDER BY COUNT(*) DESC» garantiza que los artículos recomendados sean los más populares entre los disponibles. Si no se encuentran artículos que recomendar, la función informa al usuario con un mensaje.

```
Menú de visualización con las Reviews de Amazon
-----
1 - Evolución de reviews por años
2 - Evolución de popularidad de artículos
3 - Histograma por nota (overall)
4 - Evolución de reviews a lo largo del tiempo
5 - Histograma de reviews por usuario
6 - Nube de palabras por categoría
7 - Usuarios únicos por categoría
8 - (Parte 5) Recomendación de artículos no consumidos
0 - Salir
-----
Elige una opción: 8

Recomendación de artículos no consumidos
Introduce el reviewer_id del usuario:A2HD75EMZR8QLN
1 - Video Games
2 - Toys and Games
3 - Digital Music
4 - Musical Instruments
s - salir
Selecciona una categoría:1

Artículos recomendados:
ASIN: B00BGA9WK2 // Número de reviews: 802
ASIN: B0015AARJI // Número de reviews: 652
ASIN: B007CM0K86 // Número de reviews: 517
ASIN: B002VBWIP6 // Número de reviews: 465
ASIN: B0009VXBAQ // Número de reviews: 447
ASIN: B0050SYX8W // Número de reviews: 441
ASIN: B000B9RI14 // Número de reviews: 425
ASIN: B00178630A // Número de reviews: 379
ASIN: B007XVTR3K // Número de reviews: 365
ASIN: B00005NZ1G // Número de reviews: 356
```

Figura 55: Ejemplo ejecución recomendación por popularidad

Partes opcionales

En cuanto a las partes opcionales, al haber realizado el proyecto de forma individual, se ha considerado ya muy alta la carga de trabajo que han conllevado las cinco partes obligatorias: el diseño e implementación del esquema de bases de datos, el menú de visualización con siete consultas, las cuatro opciones de Neo4J, la integración de un nuevo dataset y las visualizaciones en Tableau + Aplicación de Machine Learning.

En cualquier caso, el apartado 6.2 de esta memoria incluye una descripción detallada del modelo de Machine Learning propuesto y el diagrama de su funcionamiento, tal y como solicita el enunciado. La implementación del filtrado colaborativo basado en similitudes de Pearson (ya calculadas en la parte de Neo4J) sería el siguiente paso para avanzar hacia el apartado opcional.

Conclusiones

El proyecto ha permitido poner en práctica de forma integrada los principales sistemas de gestión de bases de datos estudiados durante el curso. La combinación de MySQL, MongoDB y Neo4J no es arbitraria. Se ha podido observar que cada sistema resuelve de forma eficiente la parte del problema para la que está mejor preparado.

MySQL ha resultado idóneo para gestionar las entidades y relaciones estructuradas del dominio, así como para ejecutar consultas agregadas y de filtrado que alimentan las visualizaciones. El modelo relacional facilita enormemente las consultas que cruzan datos de usuarios, artículos y reseñas.

MongoDB ha simplificado el almacenamiento de los campos semiestructurados, ya que el texto libre de las reseñas y los arrays de utilidad no encajan bien en columnas fijas, y el modelo documental los gestiona de forma natural. La consulta de la nube de palabras, que combina datos de MySQL y MongoDB, es un buen ejemplo de cómo los dos sistemas se pueden complementar.

Neo4J ha aportado una perspectiva diferente al análisis, representando usuarios y artículos como nodos y las valoraciones o similitudes como relaciones, permitiendo así descubrir patrones que serían complicados de expresar en SQL, como los usuarios más similares entre sí o los que consumen múltiples categorías.

Como valoración personal, a pesar de haber sido un proyecto que ha supuesto un reto técnico interesante, especialmente en la parte de Neo4J y en el cálculo de similitudes de Pearson, donde la eficiencia del acceso a MySQL para grandes volúmenes de usuarios es un factor determinante, y además haberlo realizado de forma individual. La experiencia ha reforzado la idea de que no existe un sistema de bases de datos universal, sino que la elección adecuada depende siempre de la naturaleza de los datos y de las operaciones que se quieran realizar sobre ellos.